

PaleoRigor: a skill-constrained human-in-the-loop system for reliable paleomicrobiome analysis

Author names to be completed

Affiliations to be completed

These authors contributed equally, if applicable.

*Correspondence should be addressed to: [corresponding author email]

Keywords: paleomicrobiome; ancient DNA; PaleoRigor; human-in-the-loop analysis; source-data verification; reproducible bioinformatics

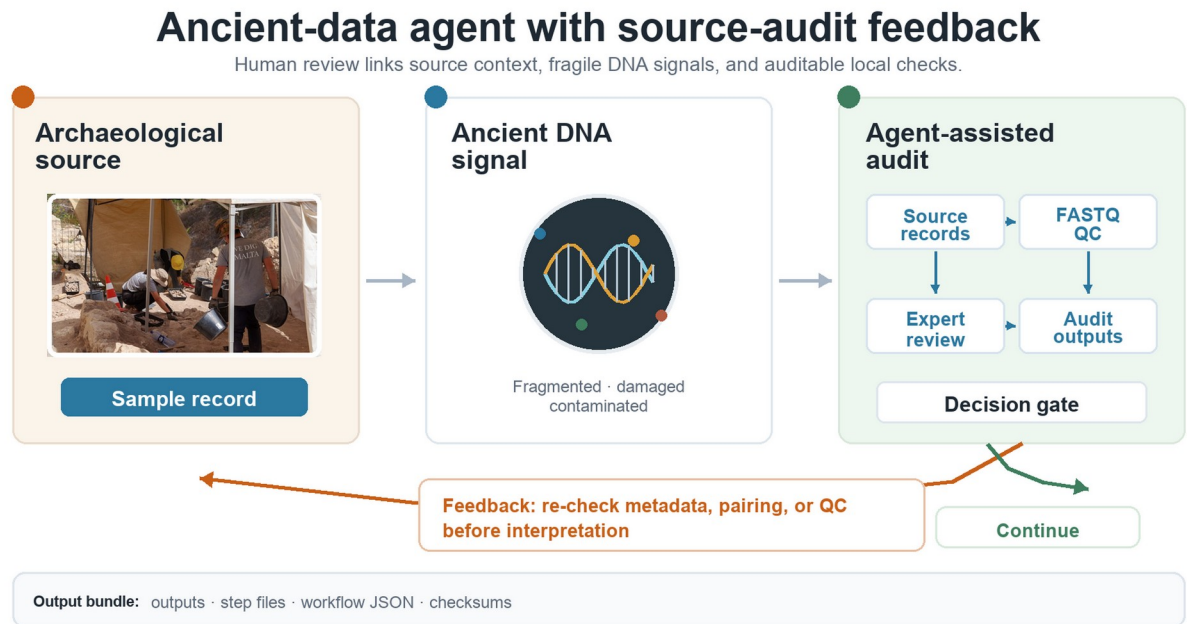
Abstract

Ancient microbial evidence can reveal the health, diet, and environments of past populations. However, this evidence is scarce, easily contaminated, and sensitive to mistakes in file selection and data handling. A standardized analysis assistant could reduce these errors and make each decision easier to inspect. Paleobiologists still lack a system that turns a research question into a reviewable analysis while keeping experts in control. We developed PaleoRigor, a human-in-the-loop system that can run only predefined, inspectable analysis modules, which we call skills. It links each requested analysis to named source files, records every data change, and asks an expert to review the proposed workflow before local execution. Two development-stage qualification rounds exposed systematic output-contract and over-planning errors. After these errors were corrected, the frozen final system completed 23 of 24 runs successfully in a new held-out evaluation (95.8%; Wilson 95% confidence interval, 79.8–99.3%), exceeding the prespecified threshold of 22 of 24. The same model with a minimal workflow prompt completed 19 of 24 runs (79.2%). PaleoRigor also matched read counts and compressed-file sizes for six public sequencing records, recorded the removal of 114 duplicate peptide sequences, and kept a clear source-audit boundary for a retraction-associated record. These results show that routine source checks and data changes can be made visible before fragile ancient evidence is interpreted. PaleoRigor does not authenticate ancient molecules, identify contamination sources, or reconstruct past microbial ecology. Instead, it gives paleobiologists an auditable starting point for deciding whether further biological analysis is justified.

Key points

- PaleoRigor addresses four risks that occur before paleobiological interpretation: unclear requests, wrong source files, hidden data changes, and claims that exceed the evidence.
- It turns these risks into reviewable decisions by linking each request to named inputs, approved steps, saved intermediate files, and expert checkpoints.
- In the final held-out evaluation, PaleoRigor passed 23 of 24 runs (95.8%), compared with 19 of 24 (79.2%) for the raw-model control; all 12 boundary decisions were handled correctly.

Graphical abstract



Graphical abstract. PaleoRigor connects archaeological source material, fragile ancient biomolecular evidence, and reviewable local analysis. The term agent-assisted audit in the figure refers to PaleoRigor; it does not imply autonomous biological interpretation. The archaeological excavation photograph was adapted from Godwin Borg, Wikimedia Commons, CC BY-SA 4.0; all other schematic elements were created by the authors. The graphical abstract shows the paleobiological use case, while Figure 1 shows the expert review and local execution steps.

Introduction

Paleomicrobiome and ancient-human microbiome studies use archaeological material to study past health, diet, and microbial exposure (Adler et al., 2013; Warinner et al., 2014, 2017; Weyrich et al., 2015, 2017). Their sequencing data are often stored in the Sequence Read Archive (SRA) and European Nucleotide Archive (ENA) (Leinonen et al., 2011; O’Cathail et al., 2025). Public access does not make a record ready for biological interpretation. Researchers must still choose the correct files, verify local copies, select suitable analyses, and record every data change. If this chain is lost, a polished result may still come from the wrong starting file. Paleobiologists therefore need a clear link between the archaeological question, the public record, and the local analysis (Figure 1; Table 4).

This link is especially important in ancient biomolecular research. Ancient DNA is scarce, fragmented, damaged, and easily affected by contamination. No single test can establish authenticity. Researchers must consider sample history, molecular damage, contamination controls, and archaeological context together (Briggs et al., 2007; Cooper & Poinar, 2000; Dabney et al., 2013; Gilbert et al., 2005; Skoglund et al., 2014; Warinner et al., 2017). Mapping, adapter removal, and damage analysis can only assess the files they receive (Jónsson et al., 2013; Schubert et al., 2012, 2016). A wrong source file can therefore pass through an otherwise successful pipeline. Source checks and computational records must come before, and remain separate from, biological authentication. Figure 4 tests source-record agreement, while Figure 5 shows why such agreement does not prove biological validity.

Standardized tools could reduce file-handling errors and make complex analyses easier for paleobiologists to inspect. Workflow engines improve reproducibility after users have chosen the inputs and steps (Di Tommaso et al., 2017; Köster & Rahmann, 2012; Sandve et al., 2013). They do not decide whether the chosen record or scientific claim is suitable. Natural-language agents can link reasoning to tool use (Yao et al., 2022), but an unrestricted agent may invent files, choose the wrong operation, or overstate routine quality-control results. The missing part is a domain-focused control layer. It should show the plan before execution and keep file-level evidence separate from biological claims.

We developed PaleoRigor as this control layer. Its planner turns a research request into a proposed workflow. Its validator checks whether the named files, formats, step references, and skills are compatible. A skill is a predefined local analysis module, not model-generated code. The local executor runs only the skills approved by the user and saves both final results and intermediate evidence. Its reporter packages the requested outputs together with the records needed to inspect the run. Paleobiologists review the plan before execution and decide what the results mean afterwards. PaleoRigor is therefore a review and execution aid, not a scientific authority (Figure 1; Tables 1 and 2).

We evaluated PaleoRigor at three linked levels. First, a repeated-run benchmark tested whether supported requests produced stable functions and outputs and whether unsuitable requests stopped safely. Second, a source-record case tested whether local sequencing files matched their intended public records. Third, a boundary case tested whether technically valid file evidence remained separate from an unsupported biological claim. A small table-curation case additionally tested whether every data change remained traceable. Together, these evaluations examine the analytical starting point, not ancient-DNA authentication or biological interpretation.

Human-in-the-loop architecture for rigorous paleomicrobiome analysis

LLM plans · experts review · local skills execute

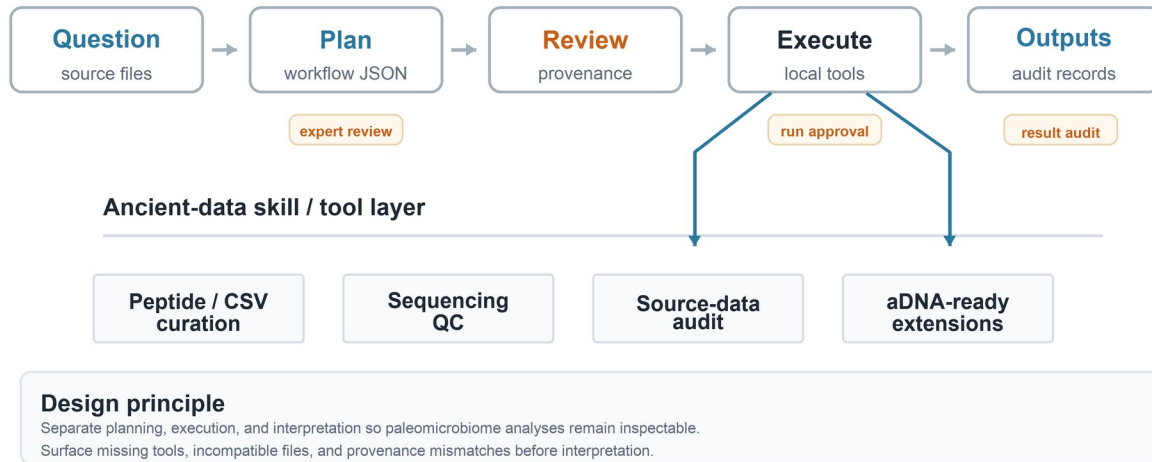


Figure 1. Human-in-the-loop design of PaleoRigor. A paleobiologist starts with a research question and available source material, such as metadata tables, sequencing files, or repository records. PaleoRigor drafts a structured workflow. Experts then review the source, file organization, task fit, and parameters before local execution. Approved skills perform limited tasks such as table cleaning, sequencing quality control, source-record comparison, and evidence packaging. Final outputs are stored separately from step records so that readers can inspect both the result and how it was produced. In the figure, LLM means large language model, and workflow JSON means a machine-readable text record of the proposed plan.

Figure 1 separates the roles of the system and the expert. PaleoRigor helps plan and run limited local operations. Paleobiologists intervene at three points: they check the question, review the workflow, and decide what the final evidence can support. Supplementary Table S4 links each checkpoint to the error it is meant to prevent.

Results

Four controls targeted four failures before paleobiological interpretation

The central problem was loss of control between a scientific question and the files eventually interpreted. We therefore tested four failure points: an unclear or unsuitable request, a wrong file or file format, an unrecorded data change, and a conclusion that exceeded the evidence. PaleoRigor paired them with expert review of the question, file and workflow validation, saved intermediate records, and an explicit interpretation boundary. Each request became an ordered workflow with named inputs, steps, parameters, outputs, and reasons before any software ran (Figure 1; Table 1).

Each control addressed a different break in this chain. Reference and compatibility checks exposed missing intermediate files and blocked operations that did not match the uploaded data. Dependency reporting prevented silent changes to the user's computer. Separate final and intermediate output folders kept data transformations visible. Expert review limited the claims that could be made from technically valid outputs. Supplementary Table S5 links observed failures to these controls, and Table 2 defines the corresponding tests.

We converted this design into three quantitative questions. First, did the frozen final system exceed the prespecified strict-success threshold on unseen supported and boundary tasks, and how did it compare descriptively with the same model under a minimal prompt? Second, did local sequencing files agree with their intended public records at the tested metadata level? Third, did unsuitable requests stop before incompatible tools or unsupported claims reached local execution? Figure 2 answers the first and third questions, Figure 4 answers the second, and Figures 3 and 5 show how traceability and claim boundaries appear in concrete outputs.

Table 1. PaleoRigor control layers and expert checkpoints. The table shows how planning, validation, execution, output storage, and interpretation are kept separate. This design supports paleomicrobiome analysis without replacing expert judgment.

Layer	Risk addressed	Control used	Expert intervention
Planner	Ambiguous user intent	Structured workflow JSON	Review proposed steps before running
Validator	Missing outputs or wrong file type	Reference and compatibility checks	Reject or revise invalid workflow
Executor	Uncontrolled code generation	Predefined skills and local tools	Approve local execution
Reporter	Opaque outputs	Final files, step files, records and checksums	Interpret results and boundaries

Six prespecified criteria made the control claims testable

We evaluated six control targets (Table 2): workflow validity, file compatibility, source agreement, traceability, dependency behavior, and repeated-run or boundary control. Each target had an explicit pass rule. Inputs had to point to available files or earlier outputs; file formats had to match skills; and public-record agreement required $\Delta reads = 0$ and $Rbytes = 1$. Traceability required final files, intermediate files, and run records. Missing software had to be reported, supported runs had to pass workflow, execution, and output checks, and boundary requests had to stop with the prespecified reason before unsafe execution.

These tests measure rigor from the source file to the reported output. They ask whether an analysis can be reviewed, repeated, and linked to the intended input. They do not test ancient-molecule authenticity, the source of contamination, microbial ecology, or evolution. Figures 3–5 keep this boundary clear by pairing each data change or quality-control value with a statement of what it does not prove.

Table 2. Engineering tests used to assess PaleoRigor. Each test represents an error seen during development and states how the system should prevent or expose it before biological interpretation.

Δ reads is the local read count minus the public read count. Rbytes is the local compressed FASTQ size divided by the public fastq_bytes value.

Evaluation target	Metric or rule	Interpretation
Workflow validity	All step inputs must reference uploaded files or earlier outputs	Prevents hallucinated pipeline references
File compatibility	Input format must match skill requirements	Prevents applying FASTQ operations to CSV files
Source agreement	Δ reads = 0 and Rbytes = 1 when public metadata are available	Confirms local files match repository records
Traceability	Final outputs plus step outputs and run records	Allows later inspection by users or supervisors
Dependency behavior	Missing tools are reported instead of auto-installed	Keeps execution transparent on user machines
Repeated-run reliability and boundary control	Supported tasks must pass all tests. Incompatible or overreaching requests must stop before unsafe execution.	Separates repeatable workflows from safe scientific boundaries.

An operational comparison defined what PaleoRigor adds to existing workflows

PaleoRigor does not replace specialist bioinformatics pipelines; it adds controls before and around their execution. Table 3 therefore compares operational functions rather than speed or biological accuracy. The six functions are planning validity, recorded table curation, public-record agreement, claim boundaries, output traceability, and repeated-run reliability. The raw-model arm in Figure 2 is a matched planning control using the same model and task files with a minimal workflow prompt; it is not a comparison with a complete external bioinformatics system. The three case studies then evaluate source and transformation evidence.

Table 3. Operational comparison of PaleoRigor with existing workflow practices. This is not a head-to-head speed or biological-accuracy benchmark. The measured-outcome column reports results obtained for PaleoRigor in this study. Figure 2 provides a matched planning-control comparison using the same model under a minimal prompt; the remaining comparison statements describe common differences in workflow control and were not measured as external software trials.

Workflow-control property	Measured PaleoRigor outcome	Comparison interpretation
Planning validity	Workflows were represented as explicit ordered steps with declared inputs, outputs, parameters, and validation rules before local execution.	Unlike manual command lines or unconstrained large-language-model code, the plan can be inspected before anything runs.

Workflow-control property	Measured PaleoRigor outcome	Comparison interpretation
Tabular biological data curation	The peptide validation table was curated from 5,810 rows to 5,696 non-redundant records; 114 duplicate sequence entries were removed.	The result is not only a cleaned table, but a recorded transformation with reusable outputs.
Public FASTQ source-metadata correspondence	Across six public FASTQ records, PaleoRigor-derived read counts matched ENA/SRA metadata with Δ reads = 0, and compressed-file ratios were Rbytes = 1.000.	The system quantifies correspondence with intended public records at the tested metadata level before downstream interpretation.
Retraction-associated source audit	For SRR29088443, PaleoRigor generated an auditable source-level report while limiting interpretation to file-level reproducibility.	The audit separates source-data reproducibility from disputed or retracted biological claims.
Output traceability	Each local run exports final_outputs, step_outputs, and ResearchAgent Records containing workflow JSON, manifests, checksums, timestamps, and run reports.	The evidence chain can be reviewed later by users, supervisors, or domain experts.
Repeated-run engineering reliability	Final v5: PaleoRigor 23/24 strict successes (95.8%) versus 19/24 (79.2%) for the matched raw-model control; all 12 PaleoRigor boundary decisions passed.	Shows release-level reliability and boundary control for the fixed task set, while the nonsignificant paired comparison and wide intervals limit generalization.

This operational framing leads to a specific testable claim: the frozen PaleoRigor release should complete more than 90% of prespecified supported and boundary runs while keeping every proposed action inspectable. The versioned benchmark below tests that claim and shows the matched raw-model control. Figures 3–5 then test whether the resulting records explain a table change, establish public-record agreement, and preserve an interpretation boundary.

Final held-out evaluation exceeded the prespecified reliability threshold

We used two earlier held-out qualification rounds to identify release-blocking errors. Each round contained four supported workflows and four boundary requests, repeated three times in PaleoRigor and raw-model arms. PaleoRigor completed 18 of 24 runs in v3 and 18 of 24 in v4 (75.0% in each round; Figure 2A). The failures exposed narrow but systematic problems: declared output names did not always map to canonical skill outputs, common chart-name variants were rejected, and the planner sometimes added sample-sheet validation when the referenced FASTQ files were unavailable. We retained these failed rounds as development evidence and corrected the corresponding output contracts and planning rules before freezing v5.

The v5 evaluation used new files, identifiers, values, and task wording. It again contained four supported and four boundary scenarios, with three repetitions per arm, giving 24 runs for PaleoRigor and 24 for the matched raw-model control. A run passed only if the response parsed, used all required and no

forbidden skills, passed production validation, and completed local execution; a boundary run had to stop with the prespecified reason. PaleoRigor passed 23 of 24 runs (95.8%; Wilson 95% confidence interval, 79.8–99.3%), exceeding the prespecified threshold of at least 22 of 24. The raw-model control passed 19 of 24 (79.2%; 59.5–90.8%; Figure 2B).

The absolute paired difference was 16.7 percentage points in favor of PaleoRigor. Five paired runs were successful only for PaleoRigor and one only for the raw model; the exact two-sided McNemar test was not significant ($p = 0.219$). The comparison therefore describes this fixed benchmark rather than establishing a general model-level advantage. By task class, PaleoRigor passed 11 of 12 supported workflows (91.7%) and all 12 boundary decisions (100%), whereas the raw model passed 9 of 12 supported workflows (75.0%) and 10 of 12 boundary decisions (83.3%).

Scenario-level results show where these differences arose (Figure 2C). PaleoRigor achieved 3 of 3 successes for FASTA curation, paired FASTQ quality control, peptide-table processing, and each of the four boundary scenarios. It achieved 2 of 3 for metadata-to-sample-sheet preparation. In the single failed run, the model omitted the .tsv suffix from the registered upload reference; the production validator reported a missing uploaded input and prevented execution. The raw model was least reliable for FASTA curation and file-format mismatch decisions (1 of 3 each) and passed 2 of 3 peptide-table runs.

V5 therefore met the release-specific reliability target while preserving a visible failure mode. The result does not erase the two 75.0% qualification rounds, prove universal reliability, or establish statistical superiority over the raw-model control. Instead, the version trajectory shows how failed tests exposed reproducible engineering defects, how those defects were corrected before a new freeze, and how the final release performed on a new fixed task set. Expert review remains necessary because the confidence intervals are wide and the benchmark cannot represent every archaeological data type or scientific claim.

We next repeated the frozen v5 design with DeepSeek-V4-Pro to test whether the result depended on the original V4-Flash planner. The tasks, files, prompts, validation rules, scorer, local tools, call order, and 24-run sample per arm were unchanged. PaleoRigor passed 24 of 24 Pro runs (100%; Wilson 95% confidence interval, 86.2–100%), whereas the matched Pro model with the minimal workflow prompt passed 19 of 24 (79.2%; 59.5–90.8%). The paired difference was 20.8 percentage points. All five discordant pairs favored PaleoRigor and none favored the raw-model arm; the descriptive exact two-sided McNemar p value was 0.0625. The result met the preregistered engineering criterion of at least 22 PaleoRigor successes and more successes than the matched control. Together with the V4-Flash result, it shows that the tested control-layer benefit was retained across two model configurations from the same provider; it does not establish a ranking of models or providers.

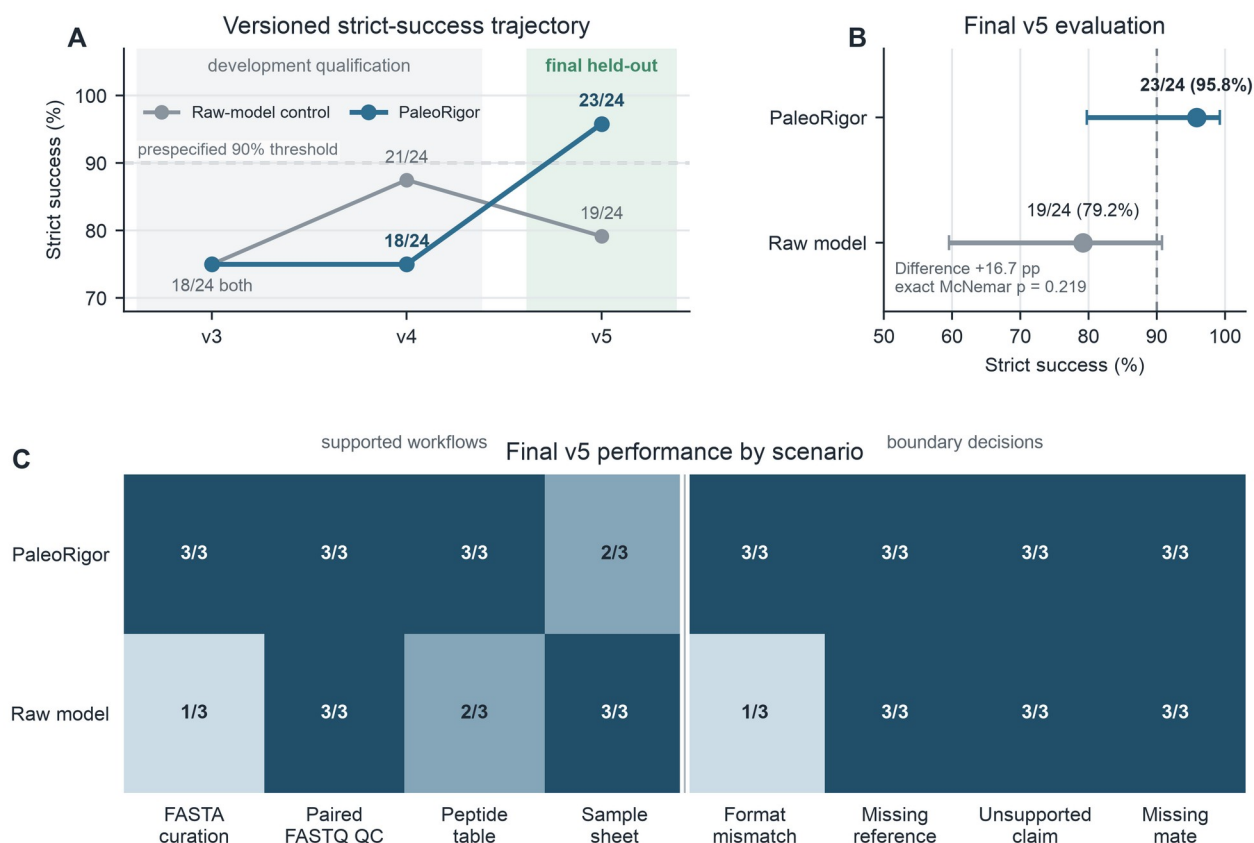


Figure 2. Versioned engineering benchmark and final held-out evaluation. A, Strict run-level success across v3, v4, and v5. V3 and v4 were retained as development-stage qualification rounds; each contained 24 PaleoRigor and 24 raw-model runs. The frozen v5 release used new files, values, identifiers, and wording. The dashed line marks the prespecified v5 threshold of more than 90%, operationalized as at least 22 of 24 successful PaleoRigor runs. B, Final v5 strict success for PaleoRigor and the same model under a minimal workflow prompt. Points show observed proportions and horizontal bars show two-sided Wilson 95% confidence intervals. The paired absolute difference was 16.7 percentage points; the exact two-sided McNemar p value was 0.219. C, V5 successes by scenario. Supported workflows comprised FASTA curation, paired FASTQ quality control, peptide-table processing, and sample-sheet preparation. Boundary decisions tested a file-format mismatch, a missing reference, an unsupported authenticity or contamination claim, and a missing paired-end mate. Each cell reports successful runs out of three.

Case study 1: a domain-neutral test of traceable table curation

The first case tested whether a simple but consequential table change could be reconstructed from the saved evidence. A peptide table was used because its expected transformation was small and exactly checkable, not because PaleoRigor was being biologically validated for peptide or paleomicrobiome research. The input contained 5,810 rows. PaleoRigor standardized the columns, checked sequence syntax, removed 114 duplicate entries, and exported 5,696 unique records (Figure 3A,B).

The saved run record linked the 114-row reduction to deduplication instead of presenting an unexplained cleaned file. Class counts and peptide lengths were retained as checks for unintended structural changes (Figure 3C,D). This result supports the traceability control defined in Table 2. It does not support a claim about peptide biology or ancient microbiomes.

Figure 3 therefore closes one part of the evidence chain: the reported output can be traced back to a named transformation. Panel A records the workflow, panel B quantifies the expected row reduction, and panels C and D check that the remaining table structure was preserved.

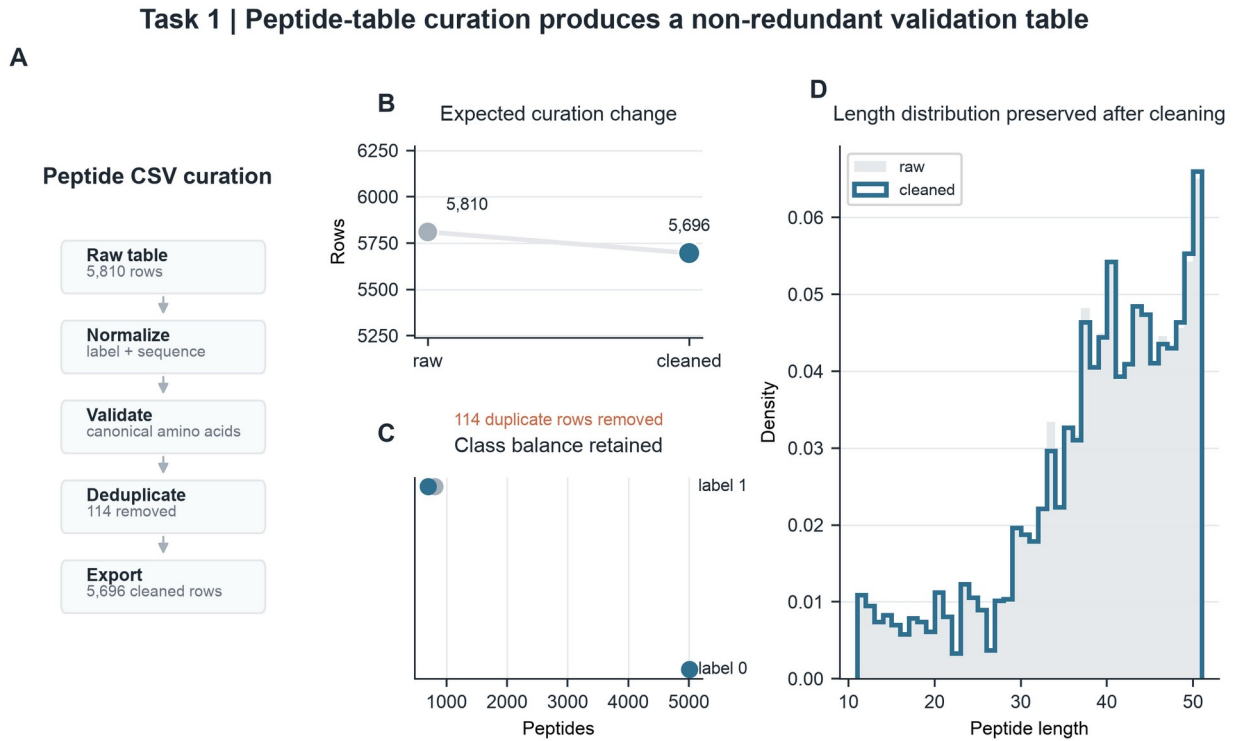


Figure 3. Peptide CSV curation. A, Steps used to process the peptide table. B, Row count before and after duplicate removal, from 5,810 input rows to 5,696 unique records. C, Counts for the two peptide labels before and after cleaning. D, Peptide-length distribution after cleaning, measured as amino-acid residues per sequence.

Case study 2: source-record agreement before ancient-DNA interpretation

The second case addressed a different failure: a technically valid FASTQ file can still be the wrong accession, mate, or download. We tested six public sequencing records related to ancient or paleomicrobiome research before any ancient-DNA interpretation. PaleoRigor checked file structure, generated FastQC reports, combined them with MultiQC, calculated file statistics with SeqKit, and compared the local files with ENA/SRA metadata (Andrews, n.d.; Ewels et al., 2016; Leinonen et al., 2011; O’Cathail et al., 2025; Shen et al., 2016). The audit retained each accession, sequencing layout, and R1/R2 organization, where R1 and R2 are paired-end mate files.

All six local records matched the public read counts exactly ($\Delta\text{reads} = 0$), and all compressed-file sizes matched the repository values ($R\text{bytes} = 1$; Figure 4A,B). This answers the second research question at the tested metadata level. The agreement does not establish byte-for-byte identity, because read counts and file sizes are not cryptographic checksums, and it does not authenticate ancient DNA. Its practical value is narrower: it reduces the risk that downstream processing begins with the wrong public file (Jónsson et al., 2013; Schubert et al., 2016).

Figure 4 shows the order of evidence. Panels A and B test agreement with public records. Panel C reports GC content, the percentage of called bases that are guanine or cytosine, as a descriptive quality-control value. Panel D summarizes record agreement across all files. The figure addresses file identity at the tested metadata level; authenticity and contamination remain separate questions.

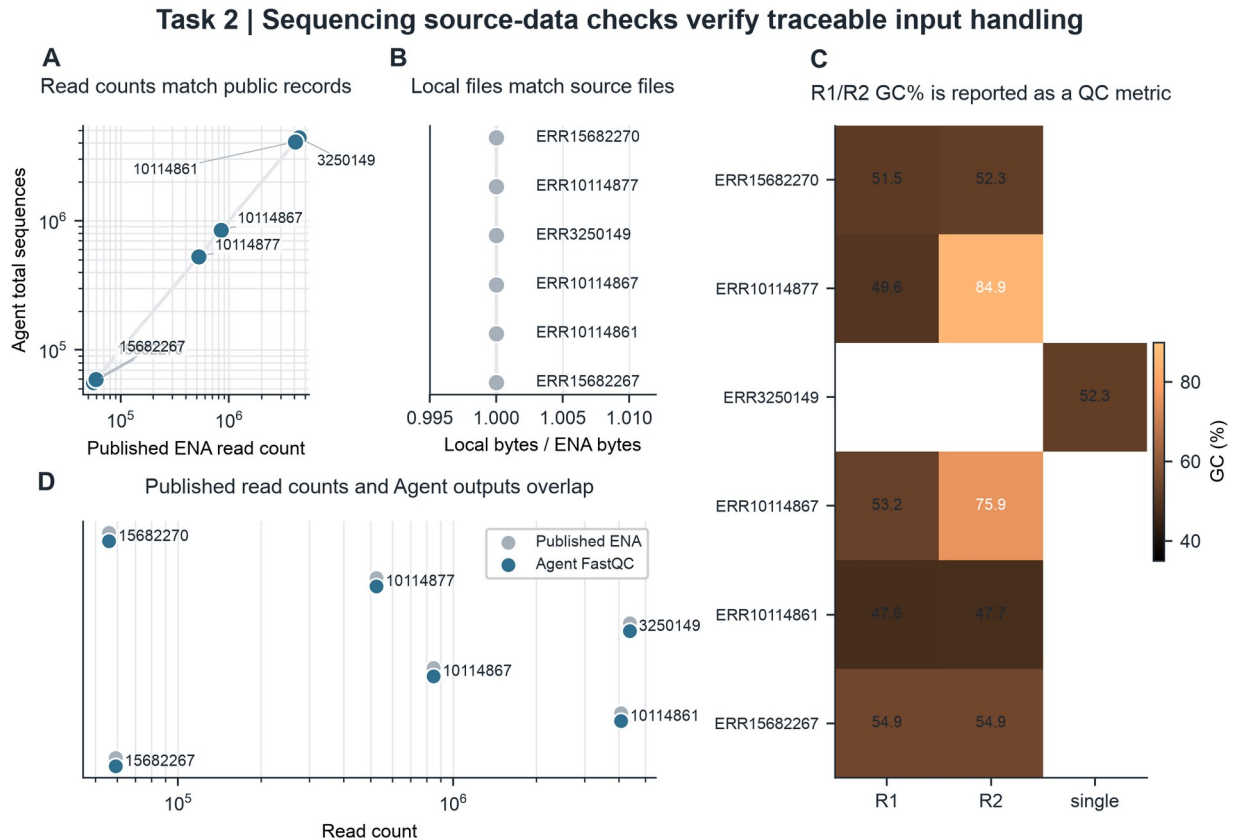


Figure 4. Agreement between local sequencing files and public records. A, Local read counts compared with ENA/SRA read counts. B, Local compressed-file sizes compared with public fastq_bytes values. Δ reads is local reads minus public reads. Rbytes is local compressed bytes divided by public fastq_bytes. C, GC content for R1, R2, or single-end files. D, Summary of record agreement across the tested files. Agreement supports a match with the intended public record, but it does not prove byte-level identity, ancient-DNA authenticity, or biological validity.

Case study 3: a negative-control test using a retraction-associated record

The third case tested the final link in the argument: technically reproducible evidence must not be treated as automatic support for a biological claim. We selected SRR29088443 from SRP508771 because its SRA study title and submitting center matched a later retracted publication record (Gao et al., 2025, 2026). The repository accession itself was not labelled as retracted, and the data came from a modern clinical microbiome study rather than an ancient-microbiome dataset. The case was therefore used as a negative control for interpretation, not as evidence about ancient biology. An expert limited PaleoRigor to file-level checks and prohibited reinterpretation of the retracted claim (Figure 5A).

Within this boundary, PaleoRigor reproduced the file-level values exactly. Each FASTQ mate contained 53,571 reads, and the combined files contained 26,517,645 bases. Both compressed-file sizes matched the public record. Quality-control reports gave mean read lengths of 244 base pairs (bp) and 251 bp and

GC contents of 54.0% and 53.6% (Figure 5B,C). These values show file-level reproducibility only. They do not support the retracted biological claim.

The result shows the intended division of responsibility. PaleoRigor reproduced the allowed file-level evidence, while the expert supplied the retraction context and defined what that evidence could not support (Figure 5D). PaleoRigor did not discover or judge the retraction autonomously. Without this checkpoint, exact metadata and clean quality-control reports could be mistaken for biological validation. The case therefore supports expert-gated claim control, whereas the boundary failures in Figure 2 show that the planner alone did not enforce that control reliably.

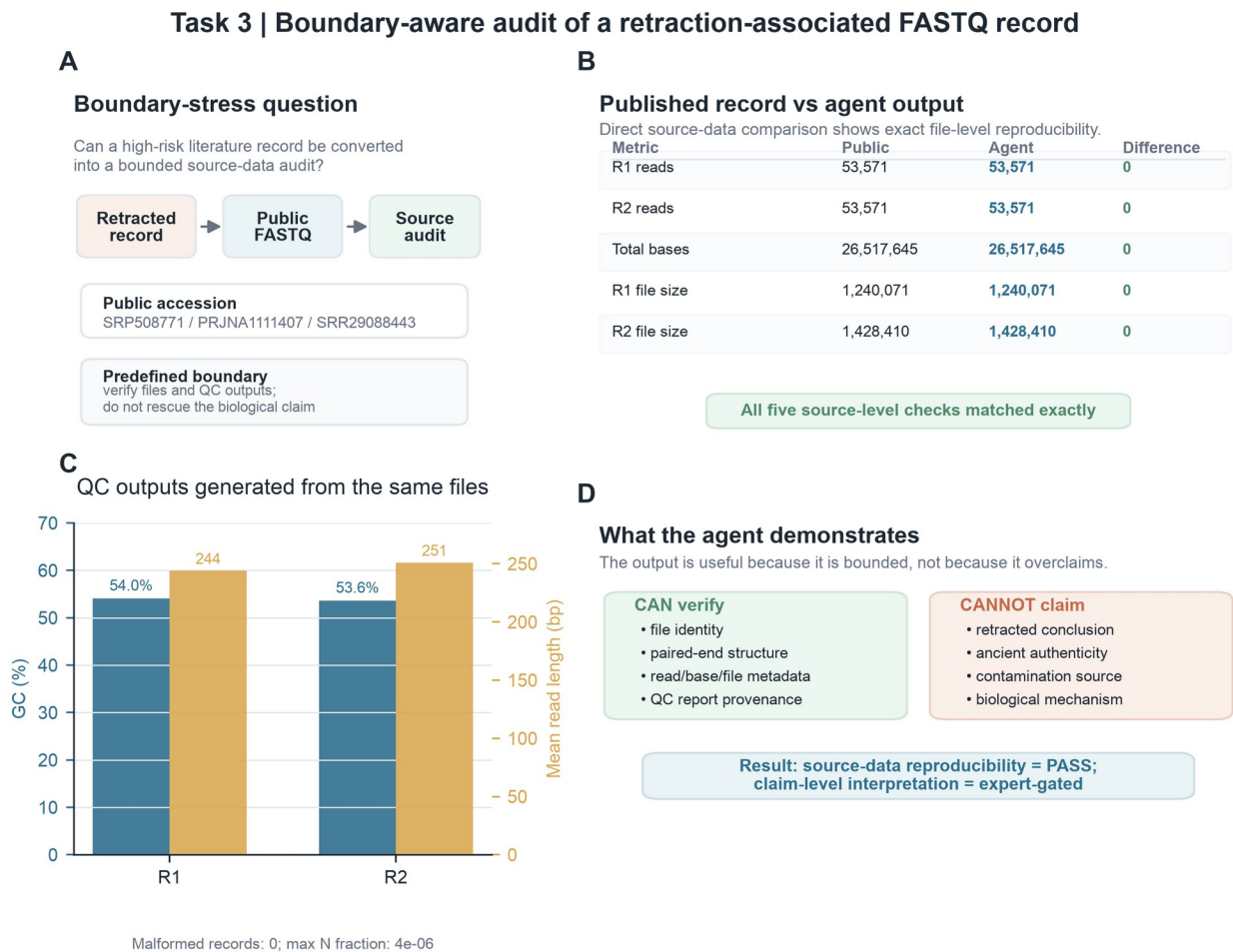


Figure 5. File-level audit of a retraction-associated FASTQ record. Retraction-associated refers to the linked publication context, not to an autonomous retraction decision by PaleoRigor. The modern clinical microbiome record was used as a negative-control test of interpretation boundaries, not as an ancient-microbiome dataset. A, The expert limited the question to a source audit. B, Public metadata and PaleoRigor outputs matched for R1 reads, R2 reads, total bases, and compressed-file sizes. R1 and R2 are paired-end mates. C, Descriptive quality-control results from the same files, including GC content, mean read length, malformed-record checks, and N-fraction. N-fraction is the proportion of bases recorded as N. D, The allowed conclusion was limited to file-level reproducibility and did not validate the retracted biological interpretation.

Discussion

PaleoRigor addresses one narrow but important pain point in paleomicrobiome research: the analytical chain can become unreliable before specialist ancient-DNA interpretation even begins. A wrong source file, an incompatible operation, or an undocumented transformation can still produce a polished output. PaleoRigor does not replace ancient-DNA methods. It makes the source, proposed steps, recorded changes, and allowed conclusion inspectable before those errors become part of a biological claim.

The first research question concerned release-level reliability. After two qualification rounds exposed repeatable implementation defects, the frozen v5 system passed 23 of 24 unseen runs and exceeded the prespecified threshold (Figure 2A,B). The lone failure was a malformed uploaded-file reference that the validator stopped before execution. This result supports reliability for the tested release and task set, not for every future request. Repeating the frozen comparison with V4-Pro produced 24 of 24 PaleoRigor successes versus 19 of 24 for its matched raw-model control. The control layer therefore retained its release-level performance across two models from one provider, although this small robustness check cannot establish cross-provider generality.

The second question concerned the connection between local files and public records. Exact read-count and compressed-size agreement across six records showed that PaleoRigor could expose the intended source relationship before downstream analysis (Figure 4). Figure 3 extends the same traceability principle to data transformations by linking a 114-row reduction to a recorded deduplication step. Together, these cases address source and handling errors without claiming ancient-DNA authenticity.

The third question concerned scientific and input boundaries. In v5, PaleoRigor returned the correct blocked decision in all 12 boundary runs, including format mismatch, missing-reference, unsupported-claim, and missing-mate scenarios (Figure 2C). This is stronger than the earlier qualification behavior, but it does not remove the need for expert judgment. Figure 5 shows the intended division: the system reports reproducible file evidence, while an expert prevents that evidence from being promoted into biological validation. Fixed rules can constrain known errors; they cannot determine archaeological meaning or molecular authenticity.

For paleobiologists, the practical contribution is therefore a checked starting point rather than an automated conclusion. PaleoRigor can reduce source-file and handling errors before adapter removal, damage analysis, host removal, taxonomic profiling, contamination analysis, or statistical interpretation begins (Jónsson et al., 2013; Schubert et al., 2016). Its evidence package allows experts to verify, revise, or reject the proposed work while preserving the connection between the archaeological question and the computational record.

The next step is to test PaleoRigor with uncertainty that better reflects archaeological samples. Controlled mixtures could contain known amounts of endogenous, environmental, handling, and laboratory material. These mixtures could test whether the system chooses suitable controls and rejects unsupported claims of authenticity (Davis et al., 2018; Eisenhofer et al., 2019; Karstens et al., 2019; Salter et al., 2014). This would test safeguards across different contamination levels, rather than only adding more executable workflows.

A larger set of negative controls is still needed. Retracted or disputed studies and low-biomass datasets affected by reagent contamination would provide useful tests (Eisenhofer et al., 2019; Salter et al., 2014). The current retraction-associated case shows that experts can set a clear interpretation boundary, while the v5 benchmark shows that predefined boundary categories can be enforced consistently. Future tests should ask whether that control generalizes to unfamiliar claims, mixed contamination sources, and incomplete archaeological metadata.

PaleoRigor will be useful only if paleobiologists can use it without losing the evidence trail. The repository therefore stores requests, workflows, intermediate files, checksums, reports, figure sources, and environment details (Supplementary Tables S1–S6). The public website (<https://ventureliu.github.io/2026summerproject/>) explains the method, evidence, and safeguards. The local browser application lets users review workflows and run analyses on their own computer. This design improves access while keeping each decision visible. Expert review remains part of the system because scientific judgment cannot be delegated to the planner.

Limitations

PaleoRigor is still a prototype control layer, not a complete ancient-DNA or paleomicrobiome platform. The evidence supports a 95.8% strict-success rate for the frozen v5 task set, recorded table changes, and agreement between local files and public records. It does not support ancient-molecule authentication, identification of contamination sources, reconstruction of microbial ecology, or superiority to complete bioinformatics pipelines. Table 3 is an operational comparison; the raw-model arm isolates the effect of the control layer on planning and execution, not biological accuracy. The peptide case tests general table handling, and the retraction-associated case lies outside ancient-microbiome research. Larger ancient-data panels, controlled contamination mixtures, and matched workflow comparisons are needed before wider use.

The final benchmark and cross-model check were small and used two model configurations from one provider in one local software environment. Each model comparison contained 24 runs per arm across eight scenario types, so the Wilson intervals remained wide. With V4-Flash, the 16.7-percentage-point paired advantage was not statistically significant (exact McNemar $p = 0.219$). With V4-Pro, the corresponding difference was 20.8 percentage points and all five discordant pairs favored PaleoRigor, but the exact test remained above 0.05 ($p = 0.0625$). These fixed-budget results support an engineering robustness criterion, not universal model superiority. Moreover, v3 and v4 both achieved only 75.0%; those rounds informed development and cannot be treated as independent support for v5. Future evaluations should preregister larger panels across independent providers, environments, genuinely unfamiliar errors, and scientific claims.

Conclusion

PaleoRigor provides paleobiologists with a reviewable path from an archaeological question and public data to a traceable local analytical starting point. The frozen final system passed 23 of 24 held-out runs (95.8%), including all 12 boundary decisions, while the matched raw-model control passed 19 of 24. Six public sequencing records matched repository read counts and compressed-file sizes, and 114 removed

duplicate rows remained explicitly documented. These findings support PaleoRigor as an auditable evidence-preparation layer, not as an autonomous interpreter of ancient biology. Its broader value will depend on larger independent benchmarks, stronger contamination controls, ancient-DNA-specific checks, and community-maintained skills. With those additions, PaleoRigor could help paleobiologists examine fragile microbial evidence while keeping sources, transformations, uncertainty, and expert judgment visible.

Methods

Agent workflow representation

A user request was represented as an ordered workflow W containing n executable steps:

$$W = \{s_1, s_2, \dots, s_n\}$$

Each step s_i was defined by a skill identifier k_i , input references I_i , output declarations O_i , parameters θ_i , and a natural-language reason r_i :

$$s_i = (k_i, I_i, O_i, \theta_i, r_i)$$

This format reduces ambiguity when a request is converted into a workflow. It turns free text into a record of source files, outputs, parameters, and reasons before execution. A skill identifier names a predefined, inspectable analysis module; it is not a command invented by the planning model. The design follows established reproducible-workflow principles and adds a natural-language review step (Di Tommaso et al., 2017; Köster & Rahmann, 2012; Sandve et al., 2013; Wilkinson et al., 2016). Figure 1 links this formal workflow to Plan, Review, Execute, and Outputs. Table 1 assigns responsibility at each stage.

Workflow validation

The validator evaluated a proposed workflow W against the available uploaded files F and the available skill registry K :

$$V(W, F, K) \rightarrow \{0, 1\}$$

A workflow was accepted only when three rules passed. Each step had to use an available skill. Each input had to point to an uploaded file or an earlier output. The input format also had to match the selected skill:

$$I_i \subseteq F \cup O_{<i}$$

$$\text{format}(I_i) \in \text{compatible}(k_i)$$

These checks address the pre-execution problem described in the Introduction. Missing outputs, inconsistent references, and wrong file types are reported before a tool runs. Supplementary Table S5 links each error type to its control. Format and reference checks do not detect unsupported scientific claims, so claim boundaries were tested separately.

Execution environment and implemented skills

Validated steps were sent to predefined local skills rather than model-generated shell commands. A skill wrapper is the inspected code that connects a workflow step to a local analysis tool and records its inputs, parameters, outputs, warnings, and errors. Table skills standardized peptide CSV files, checked sequences, removed duplicates, and exported cleaned tables. Sequencing skills supported file detection, pairing checks, FastQC, MultiQC, SeqKit statistics, and source audits. External tools ran only from configured environments, and missing software was reported instead of installed automatically. This limits execution to operations that can be inspected. Figures 3–5 show the evidence produced by these operations (Andrews, n.d.; Ewels et al., 2016; Shen et al., 2016; Supplementary Table S6).

Human-in-the-loop checkpoints

Experts reviewed the work at three points. At intake, they checked whether the request matched the scientific question. Before execution, they approved, changed, or rejected the workflow and its dependencies. After execution, they reviewed final outputs together with intermediate records and decided whether further biological analysis was justified. Figure 1 and Supplementary Table S4 show these checkpoints and the errors they are intended to prevent.

Scope and non-goals

PaleoRigor was designed to control workflows and organize evidence. It was not designed to interpret a paleomicrobiome by itself. It can check public-record metadata, file compatibility, workflow references, quality-control outputs, run records, and source links. It cannot prove byte-level file identity from metadata, authenticate ancient molecules, locate contamination, infer microbial ecology, validate retracted claims, or decide archaeological meaning. This boundary is part of the method: Figure 5D shows the allowed conclusion, and Task 6 tests whether the system keeps that limit.

Engineering benchmark protocol

The versioned benchmark used a fixed paired design. Each round contained eight scenarios: four supported workflows and four boundary requests. Each scenario was repeated three times in both a PaleoRigor arm and a raw-model arm, producing 24 runs per arm and 48 API calls per round. The arms used the same model configuration and alternating call order. The raw-model arm received the workflow schema, uploaded-file summaries, and available-skill catalogue with a minimal planning prompt. The PaleoRigor arm additionally received the staged workflow, file-compatibility, prerequisite, output-contract, and scientific-boundary rules used by the application. Supported workflows executed locally; boundary requests were never executed.

V3 and v4 were retained as failed qualification rounds after PaleoRigor achieved 18 of 24 strict successes in each. Their failures were used for development but not relabelled as confirmatory successes. Before v5, we corrected canonical output mapping, FASTA/FASTQ suffix handling, peptide-statistics output contracts, common peptide-chart aliases, and metadata-only sample-sheet planning. We then froze the v5 manifest, files, prompts, scorer, threshold, and code before any v5 API call. V5 used new files, identifiers, values, and wording. The sample size remained fixed, with no selective reruns or replacement of failures.

For each run r and required criterion d , q_r, d was 1 when the run passed the stated rule and 0 when it failed. Supported-run criteria covered parsing, required and forbidden skills, production workflow validation, and local execution. Boundary runs required a valid blocked response with the prespecified reason code. A run passed only when every relevant rule passed:

$$R_r = \prod_{d \in D_r} q_r, d$$

For task t with n_t repeated runs, the pass rate was calculated as:

$$\hat{p}_t = (1/n_t) \sum_{r \in t} R_r$$

We report two-sided Wilson 95% confidence intervals for the strict-success proportions. The prespecified v5 primary rule was at least 22 of 24 PaleoRigor successes, corresponding to more than 90%. The paired PaleoRigor–raw difference was summarized in percentage points and with an exact two-sided McNemar test; this secondary test was descriptive because 24 pairs provide limited power. Every stored v5 label was independently recomputed from the retained completion, workflow or blocked decision, production-validation result, and execution record. All 48 labels matched. Manifests, run-level scores, the summary JSON, and Figure 2 source data are stored under analysis/benchmark_v3, analysis/benchmark_v4, analysis/benchmark_v5, and Final data/manuscript_figures/benchmark_v5.

For the cross-model robustness check, we reused the frozen v5 manifest and changed only the requested model from DeepSeek-V4-Flash to DeepSeek-V4-Pro. The same eight scenarios, three repeats, two arms, prompts, temperature (0), thinking mode, JSON response mode, validation and scoring rules, local execution environment, and fixed denominator were retained. The Pro-specific hypothesis and decision rule were committed before the first formal call. It was supported only if Pro/PaleoRigor achieved at least 22 of 24 strict successes and exceeded Pro/raw. All 48 Pro labels were independently recomputed from retained completions, decisions, workflows, validation records, and executions; all matched and no API call failed. The already observed Flash data were used only as context.

Source-level agreement metrics

Read-count and compressed-file-size agreement were calculated against public ENA/SRA metadata:

$$\Delta_{\text{reads}} = N_{\text{agent}} - N_{\text{public}}$$

$$R_{\text{bytes}} = B_{\text{local}} / B_{\text{public}}$$

A file matched its public record when $\Delta_{\text{reads}} = 0$ and $R_{\text{bytes}} = 1$. Δ_{reads} is the local read count minus the public read count. R_{bytes} is the local compressed FASTQ size divided by public fastq_bytes. The run record also kept the accession, library layout, and mate organization. R1 and R2 are paired-end mate files, and single is a single-end file. These measures test agreement with a repository record, not biological claims. Figure 4 applies them to the public source panel, and Figure 5B applies them to the boundary case.

FASTQ quality-control summaries

FASTQ summaries included GC percentage, mean read length, and the fraction of bases recorded as N. For a FASTQ file with N reads and read lengths L_i , the measures were:

$$GC(\%) = (G + C) / (A + T + G + C + N) \times 100$$

$$\bar{L} = (1/N) \sum_j L_j$$

$$fN = N_{bases,N} / N_{bases,total}$$

FastQC and MultiQC generated standard quality-control files (Andrews, n.d.; Ewels et al., 2016). SeqKit produced short file-level summaries used in the figures (Shen et al., 2016). GC content and mean read length in Figures 4C and 5C describe the files; they are not evidence of ancient-DNA authenticity.

Dataset-level evaluations and rigor criteria

The evaluation included three versioned engineering-benchmark rounds and three dataset-level case studies (Table 4). V3 and v4 were development-stage qualification rounds; v5 was the frozen final held-out evaluation. The peptide table tested recorded data changes. The six-record FASTQ panel tested agreement between local files and public records. The retraction-associated record tested whether file-level evidence stayed separate from a disputed biological claim. Measured results included strict workflow success, boundary handling, duplicate removal, table structure, read counts, compressed-file sizes, and descriptive quality-control values.

Table 4. Datasets and measured results used in the evaluation. The table lists each input, its public source, the PaleoRigor task, and the measured output or required boundary. Public accessions identify the source data. The reported values show what was reproduced without making a biological claim.

Evaluation dataset	Input and original source	PaleoRigor task	Quantitative readout or expected boundary
Peptide CSV curation	Headerless local Validation.csv peptide table; 5,810 rows. Project repository: https://github.com/Verture-Liu/2026summerproject	Normalize columns, validate peptide sequences, remove duplicate sequences, compute summaries and export a cleaned table.	5,696 non-redundant records exported; 114 duplicate sequence entries removed; label balance and peptide-length distribution retained as figure checks.
Public FASTQ source-QC panel	Six public sequencing records: ERR15682270, ERR10114877, ERR3250149, ERR10114867, ERR10114861 and ERR15682267. Original ENA records: https://www.ebi.ac.uk/ena/browser/view/ERR15682270 ; https://www.ebi.ac.uk/ena/browser/view/ERR10114877 ; https://www.ebi.ac.uk/ena/browser/view/ERR3250149 ; https://www.ebi.ac.uk/ena/browser/view/ERR10114867 ; https://www.ebi.ac.uk/ena/browser/view/ERR10114861 ; https://www.ebi.ac.uk/ena/browser/view/ERR15682267	Confirm local files match intended ENA/SRA records before downstream analysis; generate FastQC, MultiQC and seqkit summaries.	PaleoRigor-derived read counts matched public metadata with $\Delta reads = 0$; local compressed-file ratios were Rbytes = 1.000; GC content and mean read length were reported as QC metrics.

Evaluation dataset	Input and original source	PaleoRigor task	Quantitative readout or expected boundary
Retraction-associated boundary audit	Paired FASTQ record SRR29088443 from SRP508771 / PRJNA1111407; 53,571 reads per mate and 26,517,645 total bases. Original SRA record: https://www.ncbi.nlm.nih.gov/sra/SRR29088443 ; BioProject record: https://www.ncbi.nlm.nih.gov/bioproject/PRJNA1111407	Test whether a problematic literature-linked record can be converted into a source-data audit without validating the retracted biological claim.	R1 reads, R2 reads, total bases, R1 file size and R2 file size matched the public record exactly; biological interpretation was treated as expert-gated.
Engineering benchmark	Versioned v3–v5 benchmark; each round used eight scenarios, three repeats, and matched PaleoRigor/raw-model arms (24 runs per arm).	Test strict workflow success, local execution, input/scientific boundaries, and the effect of the PaleoRigor control layer under fixed model settings.	V3: 18/24; v4: 18/24; frozen v5: PaleoRigor 23/24 (95.8%) and raw model 19/24 (79.2%). V5 boundary decisions: 12/12.

Output organization, public website and local application

Each local run created a dated result folder. `final_outputs` contained the files requested by the user. `step_outputs` contained intermediate files. ResearchAgent Records contained workflows, manifests, checksums, times, and reports. This structure keeps both the requested result and the evidence used to produce it. The public website (<https://verture-liu.github.io/2026summerproject/>) explains the method and installation process; it does not accept research data or run workflows. Real analyses run in the local browser application, where users review the workflow, approve execution, monitor progress, and inspect the report.

Reproducibility bundle and supplementary materials

Each case was stored as a reproducibility package with final outputs, intermediate files, and run records. Large public sequencing files were not copied into the repository. Their accessions and metadata allow readers to retrieve the same ENA/SRA records. The repository stores processed outputs, workflows, quality-control summaries, manifests, checksums, and plotting scripts. It also stores the v3–v5 benchmark manifests, the preregistered V4-Pro robustness run, run-level scores, summary files, independent verification reports, and the Figure 2 plotting script and source data. Supplementary Tables S1–S3 link manuscript claims to these files.

The supplementary package contains six evidence tables, workflow records, run reports, source manifests, and figure data with plotting scripts. It separates public data, local inputs, intermediate files, final outputs, and manuscript figures. Each reported value can therefore be traced to a workflow, source record, or script.

Data Availability

All evaluation datasets are public records or small example files in the project workspace. Table 4 lists their identity, input type, role, and measured results. Supplementary Table S1 links final outputs, intermediate files, and run records to repository folders. Supplementary Tables S2–S6 link claims, expert actions, errors, and software settings to their evidence. The v3 and v4 qualification records and the frozen v5 manifest, run-level table, summary JSON, verification report, plotting script, source-data CSV, and figure exports are stored under `analysis/benchmark_v3`, `analysis/benchmark_v4`, `analysis/benchmark_v5`, `docs/science-superpowers`, and `Final data/manuscript_figures/benchmark_v5`. The V4-Pro requests, raw completions, workflows or blocked decisions, execution records, scores, cross-model summary, and verification report are stored under `analysis/benchmark_multimodel/v4_pro`.

Data provenance policy

The project keeps six evidence layers: public records, local inputs, intermediate files, final outputs, ResearchAgent run records, and manuscript figure sources. Public FASTQ files are identified by repository accessions rather than copied. Intermediate files support step-by-step review, while final_outputs contains only requested or manuscript-relevant files. Supplementary Tables S1–S3 link these layers to result folders, workflows, and manuscript claims.

Code Availability

The public website is available at <https://verture-liu.github.io/2026summerproject/>, and the PaleoRigor code is available at <https://github.com/Verture-Liu/2026summerproject>. The website explains the method, evidence, safeguards, and local installation. It does not accept research files or run analyses. The repository contains source code, skills, example inputs, evaluation outputs, figures, workflows, reports, and source manifests. Before submission, a fixed release and archived commit will be cited and deposited in Zenodo. These materials let readers inspect the evidence behind Figures 1–5.

Supplementary Information

The Supplementary Information contains six evidence tables and three file groups. Supplementary Table S1 links each case to final outputs, intermediate files, and run records. Supplementary Tables S2–S6 cover case evidence, claim-to-evidence links, expert checkpoints, error controls, and software settings. Supplementary File 1 contains workflows, reports, manifests, and checksums. Supplementary File 2 contains figure data and scripts. Supplementary File 3 contains example final_outputs and step_outputs. Together, these materials support Figures 1–5 and Tables 1–4 without making the main text too long.

Supplementary Table S1. Links to final outputs, intermediate files, and run records. final_outputs contains requested result files. step_outputs contains files produced by each workflow step. ResearchAgent Records contains workflow JSON, manifests, timestamps, checksums, and reports. Each row links a case or sequencing record to the repository folder used to review its evidence trail.

Validation item	Result folder	GitHub result-folder URL	Evidence role
Case 1: peptide CSV curation	Final data/1	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/1	Cleaned CSV, peptide statistics, charts, normalization/validation/deduplication step outputs, workflow manifest and report. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2a: ERR15682270 R1	Final data/2	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/2	FASTQ file typing, FastQC/MultiQC outputs, quality-gate report and run records for mate R1. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2b: ERR15682270 R2	Final data/3	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/3	FASTQ file typing, FastQC/MultiQC outputs, quality-gate report and run records for mate R2. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2c: ERR10114877 paired FASTQ	Final data/group 4-5 new	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/group%204-5%20new	Paired-end FASTQ QC outputs and source-record audit records after rerun with corrected inputs. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2d: ERR3250149 single FASTQ	Final data/6	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/6	Single-end FASTQ QC outputs, source-record checks and local run records. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2e: ERR10114867 paired FASTQ	Final data/group 7 new	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/group%207%20new	Paired-end FASTQ QC outputs and source-record audit records after rerun with corrected inputs. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.

Validation item	Result folder	GitHub result-folder URL	Evidence role
Case 2f: ERR10114861 paired FASTQ	Final data/8	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/8	Paired-end FASTQ QC outputs, metadata checks and run records. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 2g: ERR15682267 paired FASTQ	Final data/9	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/9	Paired-end FASTQ QC outputs, metadata checks and run records. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Case 3: retraction-associated audit	Final data/retracted_result	https://github.com/Verture-Liu/2026summerproject/tree/main/Final%20data/retracted_result	Boundary-aware source-data audit, seqkit/FastQC/MultiQC summaries, intermediate step outputs and run records. Subfolders in the URL contain final_outputs, step_outputs and ResearchAgent Records.
Engineering benchmark	analysis/ benchmark_v3; analysis/benchmark_v4 ; analysis/benchmark_v5	https://github.com/Verture-Liu/2026summerproject/tree/main/analysis/benchmark_v5	Versioned manifests, preregistrations, run-level scores, summary JSON, verification report, and Figure 2 source data; complete raw run bundles are retained in the project archive.

Supplementary Table S2. Evidence files for each validation workflow. The table links each case to its natural-language request, workflow records, final outputs, and intermediate files.

Case	Task and workflow evidence	Final-output evidence	Intermediate evidence
Case 1 peptide CSV curation	Natural-language peptide-table curation request; workflow JSON and report in ResearchAgent Records.	cleaned CSV, statistics and charts; GitHub result-folder URL listed in Supplementary Table S1	normalization, validation, statistics, deduplication and export step outputs; GitHub result-folder URL listed in Supplementary Table S1
Case 2 public FASTQ source-QC panel	FASTQ source-check and QC requests for ERR15682270, ERR10114877, ERR3250149, ERR10114867, ERR10114861 and ERR15682267.	FastQC/MultiQC summaries and metadata JSON files; GitHub result-folder URLs listed in Supplementary Table S1	per-record step_outputs and ResearchAgent Records; GitHub result-folder URLs listed in Supplementary Table S1
Case 3 retraction-associated audit	Boundary-aware audit request for SRR29088443; workflow report documents that source-data reproducibility rather than biological validity was tested.	audit summaries and QC outputs; GitHub result-folder URL listed in Supplementary Table S1	file detection, FastQC, MultiQC, seqkit and run records; GitHub result-folder URL listed in Supplementary Table S1
Engineering benchmark	Versioned v3–v5 manifests, preregistrations, raw completions, workflows or blocked decisions, validation/execution records, and independent scoring outputs.	v5 run_level.csv, summary.json, verification_report.md, Figure 2 source data, and SVG/PDF/PNG/TIFF exports.	V3–v5 records are under analysis/benchmark_v3 , analysis/benchmark_v4 , analysis/benchmark_v5 , and Final data/manuscript_figures/benchmark_v5.

Supplementary Table S3. Links between manuscript claims and evidence. Each claim is paired with the file or record that supports it.

Manuscript claim	Primary evidence file or record	Figure/Table location	Repository route
Peptide table curation removed 114 duplicate sequence entries and exported 5,696 non-redundant records.	cleaned_Validation.csv, peptide_statistics.json and deduplication step outputs.	Figure 3; Table 4	Case 1 result-folder URL in Supplementary Table S1
Public FASTQ source records matched local inputs at the read-count and compressed-file-size level.	fastqc_run_metadata.json, file_type_report.json, MultiQC output and run records for Cases 2a-2g.	Figure 4; Table 4 and Supplementary Table S1	Case 2a-2g result-folder URLs in Supplementary Table S1
The retraction-associated case reproduced source-level metadata without validating the retracted biological conclusion.	seqkit_stats.tsv, fastqc_run_metadata.json, workflow report and boundary-aware audit figure source data.	Figure 5; Table 4 and Supplementary Tables S1–S2	Case 3 result-folder URL in Supplementary Table S1
The system separates user-facing final outputs from intermediate step outputs and run records.	Timestamped result folders containing final_outputs, step_outputs and ResearchAgent Records.	Supplementary Table S1	Repository result folders listed in Supplementary Table S1
Workflow validation prevents unsupported execution such as missing step outputs or incompatible file-type operations.	Workflow validation errors and development failure cases documented in run records and failure-mode audit.	Table 2 and Supplementary Table S5	ResearchAgent Records listed through Supplementary Table S1 result-folder URLs
The frozen final release exceeded the prespecified strict-success threshold, while earlier qualification failures remained visible.	v3–v5 manifests and summaries; v5 run_level.csv, summary.json, verification_report.md, and retained execution records.	Figure 2; Table 3	analysis/ benchmark_v3, analysis/benchmark_v4 , analysis/benchmark_v5 , and Figure 2 source-data folders

Supplementary Table S4. Expert checkpoints. The table shows where expert review occurs and which error each checkpoint is intended to prevent.

Workflow stage	PaleoRigor action	Human decision	Failure prevented
Task intake	Converts natural-language request into structured workflow steps with declared inputs, outputs and parameters.	Check whether the planned task matches the scientific question before execution.	Wrong task framing or use of an irrelevant skill.
Pre-execution validation	Checks file references, step-output dependencies and input-format compatibility.	Approve, revise or reject the proposed workflow.	Hallucinated step references, CSV/FASTQ mismatch or unsupported operation.
Local execution	Runs only predefined skills and reports progress, warnings and missing environment dependencies.	Decide whether to install missing tools or stop the run.	Silent auto-installation, uncontrolled code execution or hidden dependency failure.
Post-run review	Exports final_outputs, step_outputs and ResearchAgent Records.	Interpret outputs and decide whether downstream biological analysis is justified.	Treating QC/source checks as biological validation.
Boundary cases	Flags source-level reproducibility separately from claim-level interpretation.	Gate contested or contamination-sensitive conclusions for domain review.	Overclaiming from low-biomass or retraction-associated records.

Supplementary Table S5. Errors and implemented controls. Development failures were kept because they show why each safety check was needed.

Observed failure mode	How it appeared	Implemented control	Evidence route
Missing step-output reference	Generated workflow referred to outputs such as step_02:qc_zip or step_01/normalized that the validator could not resolve.	Strict step-output reference validation before execution.	workflow validation run records
Wrong input type for requested operation	FASTQ QC was requested for a CSV upload, or gzip decompression was proposed for an already FASTQ-classified file.	File-type detection and skill compatibility checks block incompatible workflows.	workflow records and file-type compatibility checks
Quality gate stopping on expected duplicates	The peptide validation file contained duplicate sequences, causing a strict gate to fail before deduplication.	Deduplication and quality-gate order clarified; warnings can be audited before downstream use.	Case 1 step_outputs and deduplication records
Tool-output directory assumption	FastQC initially failed when its output directory was not created before execution.	Skill wrapper creates required output directories and records tool metadata.	FASTQ final_outputs and tool metadata
User prompt/file-name mismatch	A rerun failed when the natural-language prompt still named files from a previous group.	Workflow review exposes uploaded-file references before local execution.	corrected rerun records

Supplementary Table S6. Software and environment details. The table lists the local software used to generate the evaluation outputs and figures. The final release should fix these versions in a tagged manifest.

Component	Version, environment and original URL	Role in study	Release note
PaleoRigor project	Repository: https://github.com/Verture-Liu/2026summerproject ; versioned release tag and commit to be assigned before submission.	Workflow planning, validation, local skill execution and output organization.	To be archived as a tagged GitHub release and DOI-linked snapshot before submission.
Python runtime for manuscript scripts	Python 3.13.9 in project virtual environment; Python: https://www.python.org/ ; pandas: https://pandas.pydata.org/ ; matplotlib: https://matplotlib.org/	Figure source-data processing and manuscript figure generation.	Used with pandas 2.3.3 and matplotlib 3.11.0.
FastQC	FastQC v0.12.1; original tool URL: https://www.bioinformatics.braham.ac.uk/projects/fastqc/	FASTQ quality-control reports for source-QC and boundary-audit cases.	Called through predefined fastq_qc skill; reports preserved in final_outputs.
MultiQC	MultiQC v1.35 in multiqc_env; original tool URL: https://multiqc.info/	Aggregation of FastQC reports into summary HTML/metadata outputs.	Missing environment is reported rather than silently installed.
SeqKit	SeqKit v2.13.0 in base environment; original tool URL: https://bioinf.shenwei.me/seqkit/	Compact FASTQ statistics and source-level summaries.	Used for read/base/length checks where available.
Repository and archive plan	GitHub repository: https://github.com/Verture-Liu/2026summerproject ; planned archive: https://zenodo.org/	Stable access to code, outputs, step files, run records and figure source data.	Replace mutable main-branch links with release-specific links before final submission.

Supplementary note: Figure generation and image provenance

All figures were produced from author-created vector schematics or Python-based plots of analytical outputs. The graphical abstract incorporates an openly licensed archaeological photograph with attribution. No figure was generated with a text-to-image model. Figure 2 was generated in Python from the retained v3–v5 benchmark summaries and run-level records. Its source-data CSV, plotting script, and editable SVG, PDF, PNG, and TIFF exports are preserved with the supplementary analysis materials.

Acknowledgments

Acknowledgments to supervisors, collaborators, and funding sources should be completed by the author.

Author contributions

Author contribution statements should be completed after the final author list is confirmed.

Competing interests

The authors declare no competing interests.

References

- Adler, C. J., Dobney, K., Weyrich, L. S., Kaidonis, J., Walker, A. W., Haak, W., Bradshaw, C. J. A., Townsend, G., Sołtysiak, A., Alt, K. W., Parkhill, J., & Cooper, A. (2013). Sequencing ancient calcified dental plaque shows changes in oral microbiota with dietary shifts of the Neolithic and Industrial revolutions. *Nature Genetics*, 45, 450–455. <https://doi.org/10.1038/ng.2536>
- Andrews, S. (n.d.). FastQC: A quality control tool for high throughput sequence data. Babraham Bioinformatics. Retrieved July 6, 2026, from <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
- Briggs, A. W., Stenzel, U., Johnson, P. L. F., Green, R. E., Kelso, J., Prüfer, K., Meyer, M., Krause, J., Ronan, M. T., Lachmann, M., & Pääbo, S. (2007). Patterns of damage in genomic DNA sequences from a Neandertal. *Proceedings of the National Academy of Sciences of the United States of America*, 104, 14616–14621. <https://doi.org/10.1073/pnas.0704665104>
- Cooper, A., & Poinar, H. N. (2000). Ancient DNA: Do it right or not at all. *Science*, 289, 1139. <https://doi.org/10.1126/science.289.5482.1139b>
- Dabney, J., Meyer, M., & Pääbo, S. (2013). Ancient DNA damage. *Cold Spring Harbor Perspectives in Biology*, 5, a012567. <https://doi.org/10.1101/cshperspect.a012567>
- Davis, N. M., Proctor, D. M., Holmes, S. P., Relman, D. A., & Callahan, B. J. (2018). Simple statistical identification and removal of contaminant sequences in marker-gene and metagenomics data. *Microbiome*, 6, 226. <https://doi.org/10.1186/s40168-018-0605-2>
- Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35, 316–319. <https://doi.org/10.1038/nbt.3820>
- Eisenhofer, R., Minich, J. J., Marotz, C., Cooper, A., Knight, R., & Weyrich, L. S. (2019). Contamination in low microbial biomass microbiome studies: Issues and recommendations. *Trends in Microbiology*, 27, 105–117. <https://doi.org/10.1016/j.tim.2018.11.003>
- Ewels, P., Magnusson, M., Lundin, S., & Käller, M. (2016). MultiQC: Summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*, 32, 3047–3048. <https://doi.org/10.1093/bioinformatics/btw354>

- Gao, W., Wang, X., Shi, Y., Wu, G., Zhou, M., & Lin, X. (2025). RETRACTED ARTICLE: Predictable regulation of gut microbiome in immunotherapeutic efficacy of gastric cancer. *Genes & Immunity*, 26, 1–8. <https://doi.org/10.1038/s41435-024-00306-2>
- Gao, W., Wang, X., Shi, Y., Wu, G., Zhou, M., & Lin, X. (2026). Retraction note: Predictable regulation of gut microbiome in immunotherapeutic efficacy of gastric cancer. *Genes & Immunity*, 27, 384. <https://doi.org/10.1038/s41435-026-00397-z>
- Gilbert, M. T. P., Bandelt, H.-J., Hofreiter, M., & Barnes, I. (2005). Assessing ancient DNA studies. *Trends in Ecology & Evolution*, 20, 541–544. <https://doi.org/10.1016/j.tree.2005.07.005>
- Jónsson, H., Ginolhac, A., Schubert, M., Johnson, P. L. F., & Orlando, L. (2013). mapDamage2.0: Fast approximate Bayesian estimates of ancient DNA damage parameters. *Bioinformatics*, 29, 1682–1684. <https://doi.org/10.1093/bioinformatics/btt193>
- Karstens, L., Asquith, M., Davin, S., Fair, D., Gregory, W. T., Wolfe, A. J., Braun, J., & McWeeney, S. (2019). Controlling for contaminants in low-biomass 16S rRNA gene sequencing experiments. *mSystems*, 4, e00290-19. <https://doi.org/10.1128/mSystems.00290-19>
- Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28, 2520–2522. <https://doi.org/10.1093/bioinformatics/bts480>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39, D19–D21. <https://doi.org/10.1093/nar/gkq1019>
- O’Cathail, C., Ahamed, A., Burgin, J., Cummins, C., Devaraj, R., Gueye, K., Gupta, D., Gupta, V., Haseeb, M., Ihsan, M., Ivanov, E., Jayathilaka, S., Kadirvelu, V., Kumar, M., Lathi, A., Leinonen, R., McKinnon, J., Meszaros, L., Pauperio, J., ... Cochrane, G. (2025). The European Nucleotide Archive in 2024. *Nucleic Acids Research*, 53, D49–D55. <https://doi.org/10.1093/nar/gkae975>
- Salter, S. J., Cox, M. J., Turek, E. M., Calus, S. T., Cookson, W. O., Moffatt, M. F., Turner, P., Parkhill, J., Loman, N. J., & Walker, A. W. (2014). Reagent and laboratory contamination can critically impact sequence-based microbiome analyses. *BMC Biology*, 12, 87. <https://doi.org/10.1186/s12915-014-0087-z>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9, e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Schubert, M., Ginolhac, A., Lindgreen, S., Thompson, J. F., Al-Rasheid, K. A. S., Willerslev, E., Krogh, A., & Orlando, L. (2012). Improving ancient DNA read mapping against modern reference genomes. *BMC Genomics*, 13, 178. <https://doi.org/10.1186/1471-2164-13-178>
- Schubert, M., Lindgreen, S., & Orlando, L. (2016). AdapterRemoval v2: Rapid adapter trimming, identification, and read merging. *BMC Research Notes*, 9, 88. <https://doi.org/10.1186/s13104-016-1900-2>
- Shen, W., Le, S., Li, Y., & Hu, F. (2016). SeqKit: A cross-platform and ultrafast toolkit for FASTA/Q file manipulation. *PLOS ONE*, 11, e0163962. <https://doi.org/10.1371/journal.pone.0163962>
- Skoglund, P., Northoff, B. H., Shunkov, M. V., Derevianko, A. P., Pääbo, S., Krause, J., & Jakobsson, M. (2014). Separating endogenous ancient DNA from modern day contamination in a Siberian

- Neandertal. *Proceedings of the National Academy of Sciences of the United States of America*, 111, 2229–2234. <https://doi.org/10.1073/pnas.1318934111>
- Warinner, C., Rodrigues, J. F. M., Vyas, R., Trachsel, C., Shved, N., Grossmann, J., Radini, A., Hancock, Y., Tito, R. Y., Fiddyment, S., Speller, C., Hendy, J., Charlton, S., Luder, H. U., Salazar-García, D. C., Eppler, E., Seiler, R., Hansen, L. H., Castruita, J. A. S., ... Cappellini, E. (2014). Pathogens and host immunity in the ancient human oral cavity. *Nature Genetics*, 46, 336–344. <https://doi.org/10.1038/ng.2906>
- Warinner, C., Herbig, A., Mann, A., Fellows Yates, J. A., Weiß, C. L., Burbano, H. A., Orlando, L., & Krause, J. (2017). A robust framework for microbial archaeology. *Annual Review of Genomics and Human Genetics*, 18, 321–356. <https://doi.org/10.1146/annurev-genom-091416-035526>
- Weyrich, L. S., Dobney, K., & Cooper, A. (2015). Ancient DNA analysis of dental calculus. *Journal of Human Evolution*, 79, 119–124. <https://doi.org/10.1016/j.jhevol.2014.06.018>
- Weyrich, L. S., Duchene, S., Soubrier, J., Arriola, L., Llamas, B., Breen, J., Morris, A. G., Alt, K. W., Caramelli, D., Dresely, V., Farrell, M., Farrer, A. G., Francken, M., Gully, N., Haak, W., Hardy, K., Harvati, K., Held, P., Holmes, E. C., ... Cooper, A. (2017). Neanderthal behaviour, diet, and disease inferred from ancient DNA in dental calculus. *Nature*, 544, 357–361. <https://doi.org/10.1038/nature21674>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv*. <https://arxiv.org/abs/2210.03629>